Assignment 4 Report

Ted Choi

Student # 20144986

https://github.com/Ted0007-123/cisc327-library-management-a2-4986

November 29th 2025

Introduction

The purpose of this assignment is to extend the Library Management System (LMS) developed in previous assignments by introducing end-to-end browser(E2E) testing and full application containerization. Assignment 4 focuses on three major components: First, automating realistic user flows using a browser-based E2E testing framework, second, packaging the entire Flask application into a Docker container to ensure consistent and reproducible execution across environments, and lastly, publishing this container to Docker Hub and verifying that the system can be pulled and executed from a remote registry. Through these tasks, this assignment emphasizes software quality assurance practices, reliability through automated testing, and modern deployment workflows using container technologies.

This report walks through each of these tasks in detail, covering the tools used, the steps taken, and the outcomes observed. Overall, the assignment provided practical insight into modern development workflows and highlighted how automated testing and containerization play a critical role in building reliable software systems.

Task 1 - Browser-Based End-to-End (E2E) Testing

1.1 Objective

The goal of Task 1 is to validate the behavior of the Library Management System (LMS) from a real user's perspective by automating two realistic browser interactions. Unlike unit tests, which verify isolated functions, end-to-end (E2E) tests ensure that the entire system including routing, templates, database operations, and form submissions works together correctly. The assignment requires implementing at least two user flows using a browser automation framework and executing them as part of the testing suite.

1.2 Tool Selection (Playwright)

For this task, the Playwright testing framework was chosen due to the following reasons:

- It supports Chromium, WebKit, and Firefox with a unified API.
- It integrates cleanly with pytest through the pytest-playwright plugin.
- It allows easy navigation, form submission, UI element selection, and DOM verification.
- It reliably launches headless browsers during automated test execution.

Playwright enables precise simulation of user actions such as clicking buttons, filling forms, submitting data, and verifying UI changes.

1.3 E2E Test Flow #1 — Adding a Book

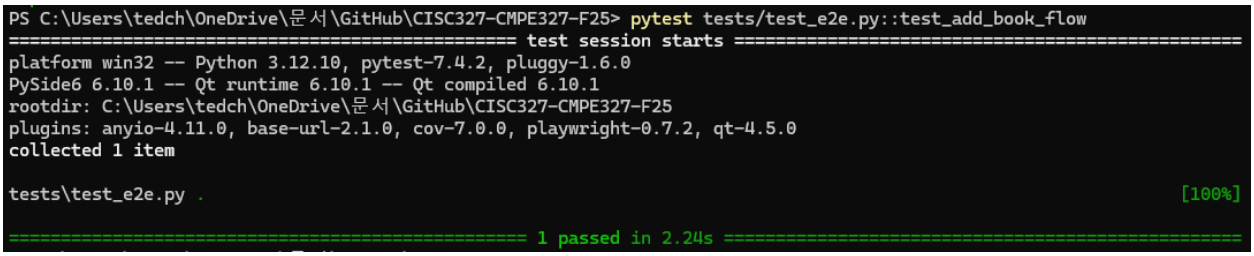
The first required user flow simulates adding a new book through the LMS UI and verifying that the catalog updates accordingly.

Steps implemented:

1. Navigate to the /catalog page.
2. Click the Add Book button.
3. Fill the book creation form with a unique title, author, ISBN, and total copies.
4. Submit the form.
5. Navigate back to the catalog page.
6. Verify that the newly added book appears in the catalog.

Outcome:

The test successfully verifies that the book submission form interacts with the service layer and database, and that the resulting catalog view correctly reflects the new entry.



1.4 E2E Test Flow #2 — Borrowing a Book

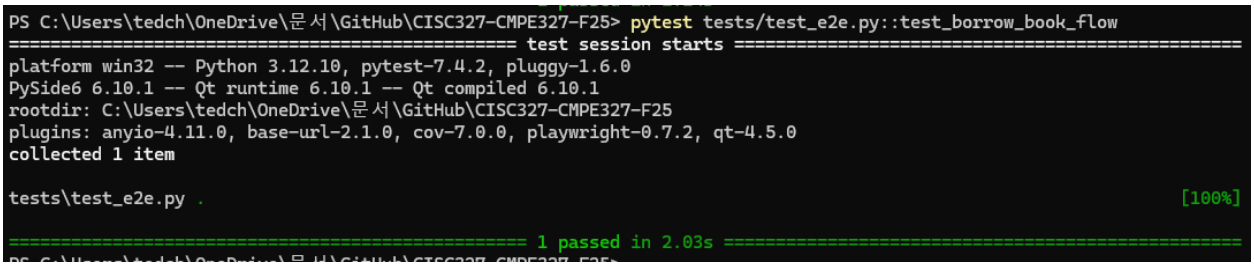
The second required user flow validates the borrowing workflow through the UI.

Steps implemented:

1. Navigate to /catalog.
2. Identify a book entry and fill in a valid patron ID.
3. Click the Borrow button.
4. Confirm that the borrowing action succeeds.
5. Verify outcome by checking:
    - Confirmation message in DOM, or
    - A change in available copies in the catalog.

Outcome:

This automated test validates the entire borrowing pipeline, including server-side validation, database updates, and UI feedback.

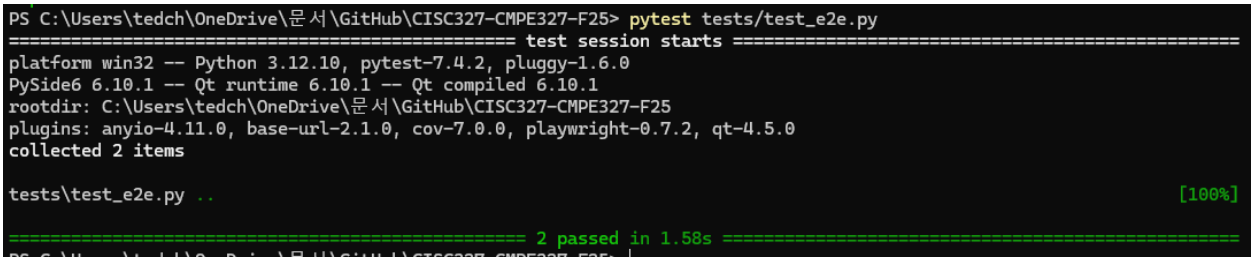


1.5 Result Summary

All flows succeeded, confirming that:

- Navigation, form submission, and DOM rendering all work as intended
- The LMS behaves correctly under full end-to-end browser simulation
- Core user journeys (Add Book, Borrow Book) operate reliably

E2E automation therefore provides strong evidence that the system functions properly from the user's point of view.



Task 2 - Application Containerization with Docker

2.1 Objective

The objective of Task 2 is to package the existing Flask-based Library Management System (LMS) into a Docker container so that it can be executed in a consistent and reproducible environment. By containerizing the application, the entire runtime environment, including Python version, dependencies, and startup command is captured in a single image. This ensures that the LMS can be run reliably on any machine with Docker installed, independent of local Python or library configurations.

2.2 Docker file Design

To containerize the LMS, a Docker file was added at the root of the project. The Docker file defines the base image, installs all required dependencies, copies the application source code, sets up environment variables, and specifies how the application should be launched inside the container.

The final Docker file is as follows:

```
FROM python:3.12-slim                  # Lightweight Python base image
WORKDIR /app                           # Set working directory
COPY requirements.txt .                # Copy dependency file
RUN pip install -r requirements.txt    # Install dependencies
COPY . .                               # Copy project code
ENV FLASK_APP=app:create_app           # Use Flask factory
EXPOSE 5000                            # Document port
CMD ["flask", "run", "--host=0.0.0.0"] # App entrypoint
```

2.3 Building the Docker Image

After writing the Dockerfile, the image is built from the project root directory using:

```
docker build -t library-app .
```

- -t library-app assigns the name library-app to the image.
- . indicates that the Docker context is the current directory (which contains the Dockerfile and project source).

A successful build produces output that ends with a message indicating that the image has been created and tagged:

```
Successfully tagged library-app:latest
```

```
PS C:\Users\tedch\OneDrive\문서\GitHub\CISC327-CMPE327-F25> docker build -t library-app .
[+] Building 0.5s (10/10) FINISHED                                                                  docker:desktop-linux
 => [internal] load build definition from Dockerfile                                                                0.0s
 => => transferring dockerfile: 379B                                                                                0.0s
 => [internal] load metadata for docker.io/library/python:3.12-slim                                                 0.3s
 => [internal] load .dockerignore                                                                                   0.0s
 => => transferring context: 177B                                                                                   0.0s
 => [1/5] FROM docker.io/library/python:3.12-slim@sha256:b43ff04d5df04ad5cabb80890b7ef74e8410e3395b19af970dcd52d7   0.0s
 => => resolve docker.io/library/python:3.12-slim@sha256:b43ff04d5df04ad5cabb80890b7ef74e8410e3395b19af970dcd52d7   0.0s
 => [internal] load build context                                                                                   0.0s
 => => transferring context: 3.46kB                                                                                 0.0s
 => CACHED [2/5] WORKDIR /app                                                                                       0.0s
 => CACHED [3/5] COPY requirements.txt .                                                                            0.0s
 => CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt                                                 0.0s
 => CACHED [5/5] COPY . .                                                                                           0.0s
 => exporting to image                                                                                              0.1s
 => => exporting layers                                                                                             0.0s
 => => exporting manifest sha256:6d258bbca65ed184b2aa6da337c278acf6517cf56eb578df856aafa53cddf213                   0.0s
 => => exporting config sha256:0bada7f95ebc4834490f20ff6adf01ef237c6b7d274185c5574c3f138ace6cae                     0.0s
 => => exporting attestation manifest sha256:3082e3ad8032225da41633bae1657fcc4d1aa7196b58cb63198a24b52c621df2       0.0s
 => => exporting manifest list sha256:88614b6324e503ccbc6a41f817b956393b3ee54efb99f954a1aedb4726f0325f              0.0s
 => => naming to docker.io/library/library-app:latest                                                               0.0s
 => => unpacking to docker.io/library/library-app:latest                                                            0.0s
```

2.4 Running the Containerized Application

Once the image is built, the LMS can be executed inside a container with:

docker run -p 5000:5000 library-app

- -p 5000:5000 maps port 5000 on the host to port 5000 inside the container.
- library-app specifies the image to run.

On startup, the container prints the standard Flask server log:

* Serving Flask app 'app:create_app'

* Debug mode: off

* Running on all addresses (0.0.0.0)

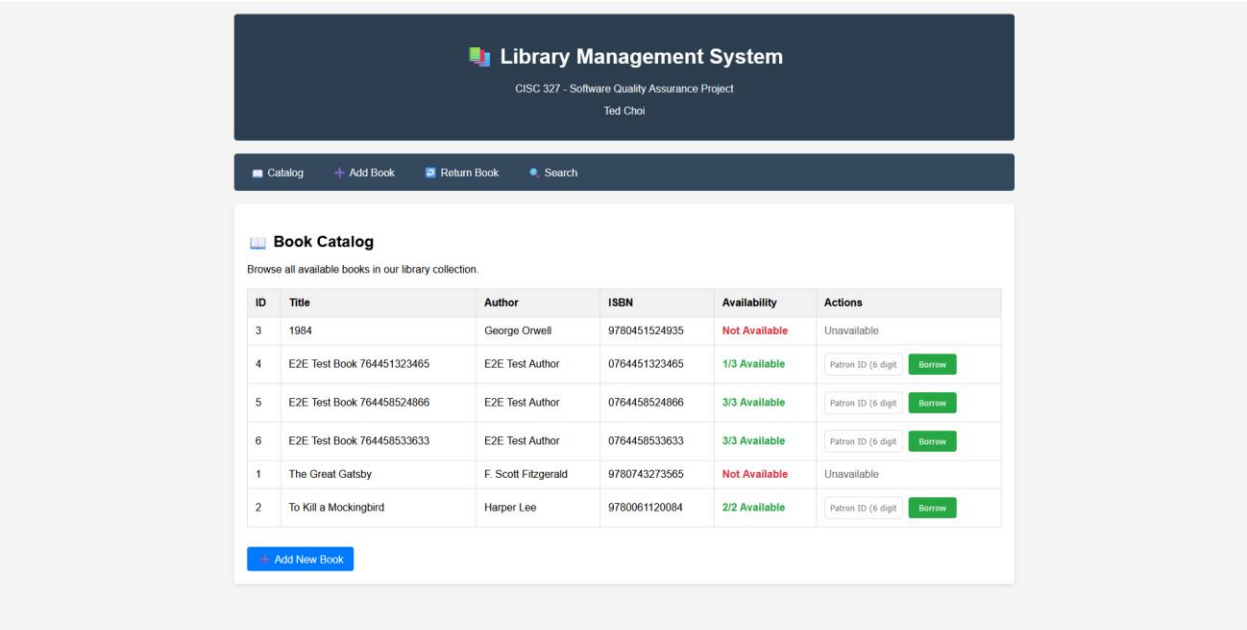
* Running on http://127.0.0.1:5000

```
PS C:\Users\tedch\OneDrive\문서\GitHub\CISC327-CMPE327-F25> docker run -p 5000:5000 library-app
 * Serving Flask app 'app:create_app'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.17.0.2:5000
Press CTRL+C to quit
```

With the container running, the application is accessible from the host machine via a web browser at:

http://localhost:5000/catalog

The catalog page loads correctly, and all features (listing books, adding books, borrowing, and returning) function as expected within the containerized environment.



2.5 Summary

Task 2 successfully containerized the Library Management System using Docker.
The Dockerfile encapsulates the Python runtime, dependencies, application code, and startup configuration in a single image. The LMS can now be built and run with just:

```
docker build -t library-app .
```

```
docker run -p 5000:5000 library-app
```

This demonstrates that the system can be reliably executed in an isolated environment, independent of the host's Python installation, which improves reproducibility and simplifies deployment.

Task 3 - Publishing and Running the Container from Docker Hub

3.1 Objective

The objective of Task 3 is to demonstrate that the containerized Library Management System (LMS) can be published to a remote container registry and later retrieved and executed from that registry. This simulates a realistic deployment workflow: instead of building the image locally on every machine, a single image is built once, pushed to Docker Hub, and then pulled and run on any host that has Docker installed. This task verifies that the Docker image is self-contained and portable.

3.2 Docker Hub Repository and Tagging

A Docker Hub account with the username ted0007 was used for this task. After successfully building the local image library-app:latest in Task 2, the image was tagged with a repository name under this Docker Hub account.

The following command assigns a new tag pointing to the Docker Hub repository:

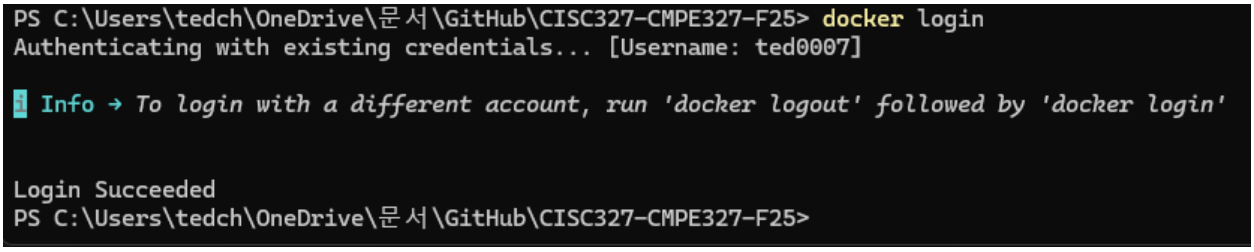
docker tag library-app ted0007/library-app:v1

- library-app refers to the locally built image.
- ted0007/library-app:v1 refers to the image name and tag in Docker Hub, where ted0007 is the Docker Hub username, library-app is the repository name, and v1 is the version tag.

Before tagging and pushing, Docker Hub authentication was performed using:

docker login

which resulted in a successful login as ted0007.



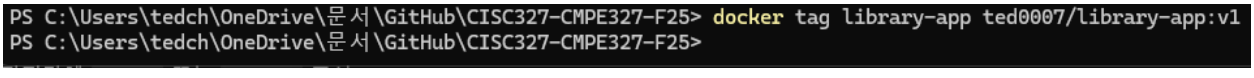
3.3 Pushing the Image to Docker Hub

Once the image was tagged, it was pushed to the Docker Hub repository using:

docker push ted0007/library-app:v1

During this process, Docker uploaded the individual image layers to the remote registry. After the push completed, the repository ted0007/library-app with tag v1 became visible in the Docker Hub web interface.

This confirms that the image is now hosted in a remote registry and can be accessed from other machines or environments.



```
PS C:\Users\tedch\OneDrive\문서\GitHub\CISC327-CMPE327-F25> docker push ted0007/library-app:v1
The push refers to repository [docker.io/ted0007/library-app]
490b9a1c25e4: Layer already exists
7288bd8a99fb: Layer already exists
0e4bc2bd6656: Layer already exists
03eb3e964b90: Pushed
0674d14a155c: Layer already exists
b7ba6d2a1fc7: Layer already exists
e6485ceba436: Pushed
dcb1eb4b8d08: Layer already exists
6c23ec7dbab4: Layer already exists
v1: digest: sha256:88614b6324e503ccbc6a41f817b956393b3ee54efb99f954a1aedb4726f0325f size: 856
```

3.4 Pulling the Image from Docker Hub

To verify that the LMS image can be retrieved from Docker Hub, the image was pulled using:

docker pull ted0007/library-app:v1

This command downloads the image layers from Docker Hub to the local machine (or to any host that does not yet have the image). If the image already exists locally, Docker verifies and reuses the existing layers; otherwise, they are freshly downloaded.

```
PS C:\Users\tedch\OneDrive\문서\GitHub\CISC327-CMPE327-F25> docker pull ted0007/library-app:v1
v1: Pulling from ted0007/library-app
Digest: sha256:88614b6324e503ccbc6a41f817b956393b3ee54efb99f954a1aedb4726f0325f
Status: Image is up to date for ted0007/library-app:v1
docker.io/ted0007/library-app:v1
PS C:\Users\tedch\OneDrive\문서\GitHub\CISC327-CMPE327-F25>
```

3.5 Running the Application from the Pulled Image

After pulling the image, the LMS was started directly from the Docker Hub image with:

docker run -p 5000:5000 ted0007/library-app:v1

- -p 5000:5000 maps port 5000 on the host to port 5000 in the container (the port on which the Flask app listens).
- ted0007/library-app:v1 explicitly uses the image that was pulled from Docker Hub.

When the container starts, the terminal shows the Flask application log:

* Serving Flask app 'app:create_app'

* Debug mode: off

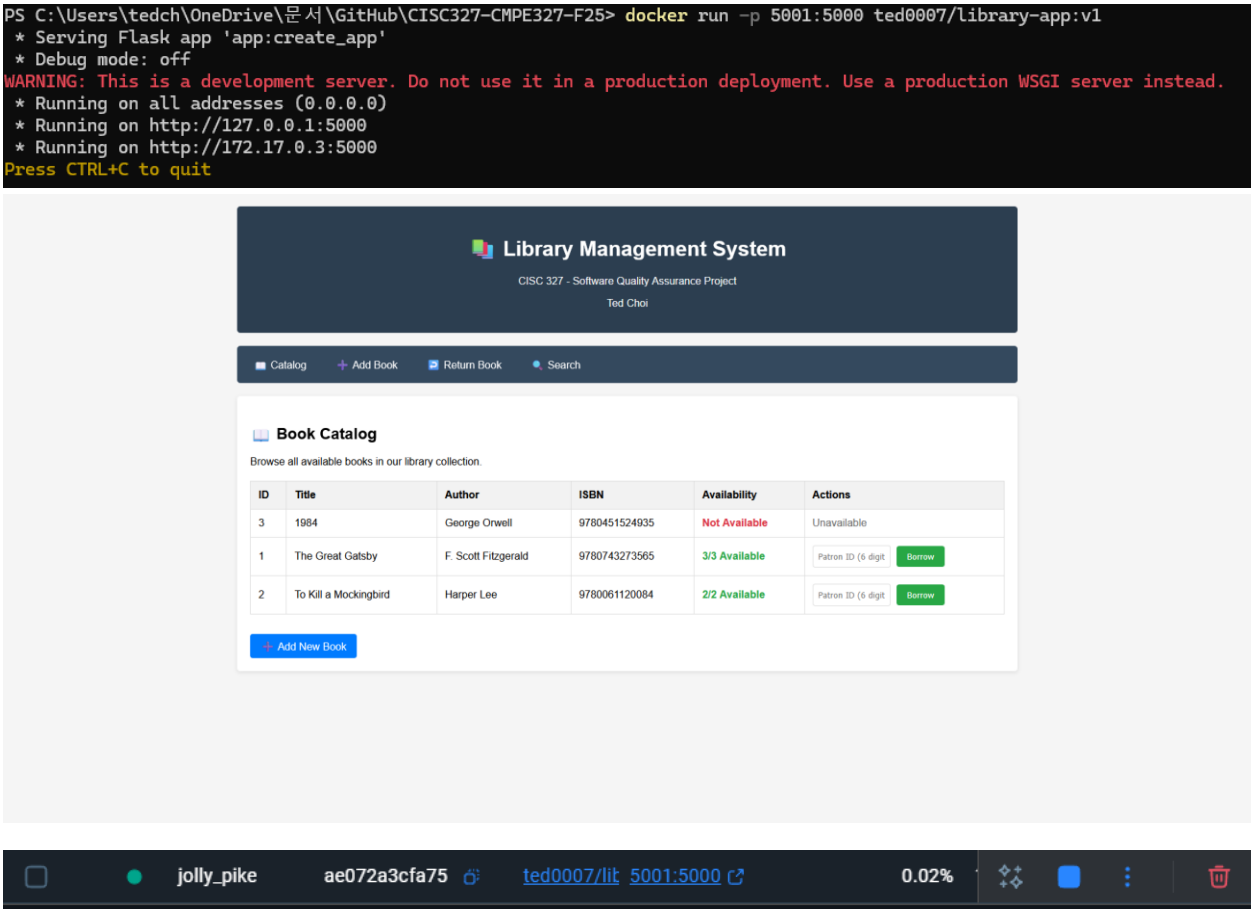
* Running on all addresses (0.0.0.0)

* Running on http://127.0.0.1:5000

With the container running, the application is again accessible from the host browser at:

http://localhost:5000/catalog

The catalog page loads correctly and all LMS functionality (viewing books, adding books, borrowing, and returning) works as expected, confirming that the pulled image is complete and executable without any additional configuration.



3.6 Summary

Task 3 demonstrates that the containerized Library Management System can be:

1. Tagged and pushed to a remote registry (Docker Hub) as ted0007/library-app:v1.

2. Pulled from Docker Hub on demand.
3. Executed directly from the pulled image using Docker.

This completes the deployment pipeline: starting from local development and testing, to containerization, to hosting the image in a public registry and running it from there. The LMS is now portable and can be run consistently on any machine with Docker installed, without having to rebuild or manually configure the environment.

5. Reflection

This assignment provided valuable hands-on experience in developing, testing, and deploying a software system using industry-standard tools. Implementing the browser-based E2E tests using Playwright reinforced the importance of validating an application from the user's perspective rather than relying solely on unit tests. Through this task, I learned how automated UI tests can catch issues related to routing, templates, and form submissions that are not visible in isolated tests.

Containerizing the application with Docker further emphasized the value of environment consistency. By packaging the entire system—including dependencies, configuration, and runtime—into a single image, the LMS became reproducible across any machine regardless of local setup. This made it clear how modern software deployment pipelines rely on containers to ensure predictable behavior in production environments.

Publishing the Docker image to Docker Hub and pulling it back onto the local machine demonstrated a full deployment workflow. This experience showed how applications can be distributed and executed easily without rebuilding, and how container registries act as reliable sources for deployment artifacts.

Overall, Assignment 4 helped me understand the relationship between automated testing, environment isolation, and deployment portability. These skills are directly applicable to real-world software engineering workflows, and this assignment strengthened my ability to develop and validate systems in a professional and reproducible manner.